Securing AI Agents with PingOne using Delegation and Least Privilege

PingOne PingGateway [this will be product chips like we have in the release notes]

Organizations are deploying AI-driven digital assistants to act on behalf of humans. To securely deploy these agents, treat them as first-class, non-human identities distinct from end users and enforce least-privilege access.

Consider a chatbot embedded in a banking app. Customers use the chatbot to retrieve account balances and make transactions. Acting on behalf of the customer, the chatbot accesses backend account resources. To secure these interactions, PingOne issues tokens and exchanges them for each agent action. PingGateway validates the tokens, acting as a security proxy in front of account resources.

You can find more details about the process flow in Securing Digital Assistants.

Goals

- Secure a digital assistant to ensure the agent only performs authorized actions on behalf of the user.
- Configure OAuth 2.0 token exchange to issue downscoped access tokens that preserve the chain of delegation using the `act` claim.
- Protect backend MCP servers and APIs using PingGateway.

What you'll do

To enable secure delegation, you'll set up the following components in PingOne and PingGateway:

- Add an OIDC client application
- Add custom resources
- Add a consent agreement
- Add an authentication policy
- Add an AI agent

- Configure PingGateway
- Start the MCP agent

Before you begin

To complete this use case, you'll need:

- Administrator access in a PingOne environment configured to work with PingGateway. Learn more in PingGateway and PingOne.
- PingGateway installed and configured with server-side SSL (TLS). The sample application isn't required for this use case. Learn more in MCP security gateway and Configure PingGateway for TLS (server-side).

  Example `admin.json` file [this will be a collapsed heading]

  This is an example `.openig/config/admin.json` connector for SSL. The user is `macuser` and the secrets folder `/Users/macuser/.openig/secrets` contains the self-signed certificate for `ig.example.com`.

```
{
  "adminConnector": {
    "host": "localhost",
    "port": 8085
  },
  "connectors": [
    {
      "port": 8080
    },
    {
      "port": 8443,
      "tls": "ServerTlsOptions-1"
    }
  ],
  "streamingEnabled": true,
  "heap": [
    {
      "name": "ServerTlsOptions-1",
      "type": "ServerTlsOptions",
```

```
      "config": {
        "keyManager": {
          "type": "SecretsKeyManager",
          "config": {
            "signingSecretId": "key.manager.secret.id",
            "secretsProvider": "ServerIdentityStore"
          }
        }
      }
    },
    {
      "name": "ServerIdentityStore",
      "type": "FileSystemSecretStore",
      "config": {
        "format": "PLAIN",
        "directory": "/Users/macuser/.openig/secrets",
        "suffix": ".pem",
        "mappings": [{
          "secretId": "key.manager.secret.id",
          "format": {
            "type": "PemPropertyFormat"
          }
        }]
      }
    }
  ]
}
```

- A deployed MCP server capable of exposing tools the agent will access, such as `banking:read_balance` and `banking:transfer`. You can use the sample MCP and agent software described in [MCP security gateway preparation](#).

  Note: The sample MCP and agent software is for a weather application, but you can adapt them for this use case if you don't already have an MCP server or agent.

Tasks

Adding a banking application

In PingOne, register the OIDC client application your customers sign on to for banking. You can find more details in Adding an application.

1. In the PingOne admin console, go to **Applications > Applications** and click the **+** icon.
2. Name the application **Banking Web App**, select **OIDC Web App** as the application type, and click **Save**.
3. On the **Configuration** tab, click the **Pencil** icon to edit configuration settings.
4. Under **Grant Type**, make sure **Authorization Code** is selected, then select **REQUIRED** in the **PKCE Enforcement** list.
5. Click **Save**.
6. Copy the **Client ID** and store it somewhere convenient. You'll use this in your web application's code.

Adding custom resources

Add custom resources that represent the services the agent will access. You can find more details in Adding a custom resource.

Note: Settings for your own custom resource will vary from those used as examples in this use case.

1. In the PingOne admin console, go to **Applications > Resources** and click the **+** icon.
2. Add the test resource:
    a. Name the resource **test**, enter `https://ig.example.com:8443/mcp` for the **Audience**, and click **Next**.
    b. Click the **Gear** icon next to the **sub** attribute, enter the expression `#root.context.requestData.subjectToken.sub`, and click **Save**.
    c. Click **+ Add** and name the attribute **act**.

d. Click the **Gear** icon next to the **act** attribute, enter the expression `(#root.context.requestData.subjectToken.may_act.sub == #root.context.requestData.actorToken.client_id)?#root.context.requestData.subjectToken.may_act:null`, and click **Save**.

e. Click **Next**, then click **+ Add Scope**.

f. Name the scope **test** and click **Save**.

g. Click the **test** resource to display the **Overview** tab.

h. Copy the **Resource ID** and **Client Secret** and store them somewhere convenient. You'll use them for PingOne Gateway configuration.

i. Close the **test** resource.

3. Add the agent resource:

   a. Click the **+** icon.

   b. Name the resource **agent**, keep the default **agent** for the **Audience**, and click **Next**.

   c. For the **sub** attribute, select **Username** in the **PingOne Mappings** list.

   d. Click **+ Add** and name the attribute **may_act**.

   e. Click the **Gear** icon next to the **may_act** attribute, enter the expression `(#root.context.requestData.grantType == "client_credentials")?null:({ "sub": #root.context.appConfig.clientId })`, and click **Save**.

   f. Click **Next**, then click **+ Add Scope**.

   g. Name the scope **agent** and click **Save**.

Adding an agreement

Add an agreement for the user's consent for agent actions. You can find more details in Agreements and Adding an agreement.

1. In the PingOne admin console, go to **User Experience > Agreements** and click the **+** icon.

2. Name the agreement **Agent Consent** and enter a description such as **Consent for a digital assistant agent**.

3. In **Reconsent Every**, select **Number of Days** and keep the default 180.
4. Click **Save**.
5. Add a language:
    a. Click **Edit Localized Content** and click the **Languages +** icon.
    b. In the **Language** list, select **English (en)** and click **Save**.
    c. In **Localized Agreement Content**, enter **I consent to allow digital assistants created by MyCompany to act on my behalf**, then click **Save**.

       The language is enabled by default.
    d. Close the language.
6. On the **Agreements** page, click the toggle to enable the **Agent Consent** agreement.

Adding an authentication policy

Add an authentication policy for the agent. You can find more details in [Adding an authentication policy](#).

1. In the PingOne admin console, go to **Authentication > Authentication** and click **+ Add Policy**.
2. Name the policy **Agent-Consent-Login**.
3. In the **Step Type** list, select **Login**, then click **+ Add step**.
4. In the **Step Type** list, select **Agreement Prompt**.
5. Select the **Agent Consent** terms of service agreement and click **Save**.

Adding an AI agent

Register the agent as an identity in PingOne and configure its authentication settings, such as scopes for the actions the agent can perform. You can find more details in Adding an AI agent. [link will be something like https://docs.pingidentity.com/pingone/ai_agents/p1_intro_to_ai_agents.html]

1. In the PingOne admin console, go to **Applications > AI Agents** and click the **+** icon.
2. Name the agent **MCP Tutorial**, enter a description such as **MCP agent for PingGateway tutorial**, and click **Save**.
3. On the **Configuration** tab, click the **Pencil** icon to edit configuration settings:

   Note: Keep all of the default settings.

   a. In **Grant Type**, select **Client Credentials**, **Refresh Token**, and **Token Exchange**.

   b. In **Redirect URIs**, enter http://localhost:3000/callback.

   c. Click **Save**.
4. On the **Resources** tab, click the **Pencil** icon to edit resource settings:

   a. Select the **agent** and **test** scopes.

   b. Click **Save**.
5. On the **Policies** tab, click **+ Add Policies**.

   a. Select the **Agent-Consent-Login** policy.

   b. Click **Save**.
6. Click the toggle next to **MCP Tutorial** to enable the agent.
7. On the **Overview** tab, copy the **Client ID** and store it somewhere convenient. You'll use it when restarting the MCP agent.

   Result: You've successfully configured PingOne to act as the authorization server.

Configuring PingGateway

PingGateway serves as the enforcement point that protects the MCP server and validates tokens.

1. In the `admin.json` file for PingGateway, enable streaming:

```
None
"streamingEnabled": true
```

Note: This is already enabled in the example `admin.json` file you set up as a prerequisite. PingGateway requires this setting for server-side events (SSE), an MCP transport option.

2. Export a `RESOURCE_SECRET_ID` environment variable with the base64 encoded client secret for the PingOne test resource:

   bash: `export RESOURCE_SECRET_ID=$(echo -n "<Resource client secret>" | base64)`

   Important: Make sure not to include a newline.

3. Add the following route to PingGateway:

   Linux

   `$HOME/.openig/config/routes/mcp.json`

   Windows

   `%appdata%\OpenIG\config\routes\mcp.json`

   Enter the `PingOne environment ID` and `PingOne resource ID` for your deployment.

```
JSON
{
  "name": "mcp",
  "condition": "${find(request.uri.path, '^/mcp')}",
  "properties": {
    "pingOneEnvID": "https://auth.pingone.com/<PingOne
Environment ID>",
    "pingOneResourceID": "<PingOne test resource ID>",
    "gatewayUrl": "https://ig.example.com:8443",
    "mcpServerUrl": "http://localhost:8000"
  },
  "baseURI": "&{mcpServerUrl}",
  "heap": [
```

```
    {
      "name": "SystemAndEnvSecretStore-1",
      "type": "SystemAndEnvSecretStore"
    },
    {
      "name": "AuditService",
      "type": "AuditService",
      "config": {
        "eventHandlers": [
          {
            "class":
"org.forgerock.audit.handlers.json.JsonAuditEventHandler",
            "config": {
              "name": "json",
              "logDirectory": "&{ig.instance.dir}/audit",
              "topics": ["access", "mcp"]
            }
          }
        ]
      }
    },
    {
      "name": "rsFilter",
      "type": "OAuth2ResourceServerFilter",
      "config": {
        "requireHttps": false,
        "scopes": ["test"],
        "accessTokenResolver": {
          "type": "TokenIntrospectionAccessTokenResolver",
          "config": {
            "endpoint": "&{pingOneEnvID}/as/introspect",
            "providerHandler": {
              "type": "Chain",
              "config": {
                "filters": [
                      {
```

```
                          "type":
"HttpBasicAuthenticationClientFilter",
                          "config": {
                            "username": "&{pingOneResourceID}",
                            "passwordSecretId":
"resource.secret.id",
                            "secretsProvider":
"SystemAndEnvSecretStore-1"
                          }
                        }
                      ],
                  "handler": "ForgeRockClientHandler"
                }
              }
            }
          }
        }
      }
    ],
    "handler": {
      "type": "Chain",
      "config": {
        "filters": [
          {
            "type": "McpAuditFilter",
            "config": {
              "auditService": "AuditService"
            }
          },
          {
            "type": "UriPathRewriteFilter",
            "config": {
              "mappings": { "/mcp": "/" }
            }
          },
          {
```

```
          "type": "McpProtectionFilter",
          "config": {
            "resourceId": "&{gatewayUrl}/mcp",
            "authorizationServerUri": "&{pingOneEnvID}/as",
            "resourceServerFilter": "rsFilter",
            "supportedScopes": ["test"],
            "resourceIdPointer": "/aud/0"
          }
        },
        {
          "type": "McpValidationFilter",
          "config": {
            "acceptedOrigins": ".*"
          }
        }
      ],
      "handler": {
        "type": "ReverseProxyHandler",
        "config": {
          "soTimeout": "20 seconds"
        }
      }
    }
  }
}
```

Features of the sample route:

- This route uses a secret obtained from an environment variable.
- PingGateway acts as an OAuth 2.0 resource server (RS) when protecting the sample MCP server.
- The McpAuditFilter audits MCP requests. PingGateway records MCP audit events in an `audit/mcp.audit.json` file.
- The UriPathRewriteFilter sends the request to the root resource of the MCP server. The MCP server expects requests at /.
- The McpProtectionFilter uses the RS configuration, extending it for MCP.

- PingGateway validates MCP requests with an McpValidationFilter.
- The ReverseProxyHandler uses a long `"soTimeout"` setting to accommodate an MCP agent receiving few or infrequent SSE updates.

4. Restart PingGateway to apply the route changes.

5. Add throttling or fine-grained access control as necessary to meet your security requirements.

   This simple route doesn't include those features.

6. Check the PingGateway log to verify the route loads successfully.

   Result: You've successfully configured PingGateway to protect the sample MCP server.

Start the MCP agent

1. In the directory where you unpacked the sample MCP agent, export the AI agent secret as an environment variable called `AGENT_SECRET`.

2. Start the sample MCP agent again with the added option `-client id`.

   You'll need the client ID from the MCP Tutorial agent. Point it to the PingGateway route for MCP requests.

```
None

bash-3.2$ export AGENT_SECRET=yjS0IWMX9LS91...
bash-3.2$ python3 sample-mcp-agent.py --client-id <PingOne AI
agent client ID> --mcp-server-url
https://ig.example.com:8443/mcp
```

   Result: You've successfully started the sample MCP agent.

Validation

With PingGateway protecting the MCP server, the sample MCP agent directs you to the authorization server to sign on as an end user and authorize access to make MCP requests.

1. Allow the agent to open a browser automatically, or:

    a. Select `'n'`.

    b. Copy the authorization URL that the sample MCP agent displays in your terminal.

    c. Paste the URL into a browser.

    The URL should look similar this:

    https://auth.pingone.com/082e56dd-1a0f-4e1d-a28e-77d51ecf1705/as/authorize?response_type=code&client_id=3fffb1bf-106c-4449-a9bd-df6fa76d8f41&redirect_uri=http%3A%2F%2Flocalhost%3A3000%2Fcallback&state=…

2. Sign on as the test user and consent to allow a digital assistant (if asked) before you close the browser tab.

    The following commands are available in the terminal where the sample MCP agent is running:

```
None
[INFO] Discovered tools [https://ig.example.com:8443/mcp]:
[INFO] - geocode: Returns a list of objects containing city
name, latitude, longitude, country, admin1 (region), and
timezone for each matching city
[INFO] - forecast_daily: Returns a multi-day weather forecast
for a given location
[INFO] - forecast_periods: Returns weather forecasts for each
representative period of the current day
[INFO] - forecast_hourly: Returns an hourly weather forecast
for the current day
[INFO] - weather_at_time: Returns the forecasted weather for a
specific time at a given location

Enter your message (or 'exit|quit|q'):
```

3. Enter a prompt and get a response from the MCP server through PingGateway, then exit the agent.

The following example uses the `forecast_daily` tool to get the daily forecast for Tokyo:

```
None
Enter your message (or 'exit|quit|q'): What is the daily
forecast for Tokyo?
Agent: The daily forecast for Tokyo is:

<MCP server response with forecast details>

Enter your message (or 'exit|quit|q'): exit
User requested exit. Goodbye!
```

The following tokens are used in the transaction:

- Actor token: The AI agent's access token.
- Subject token: The end user's access token. The `may-act` claim asserts that the agent is allowed to act on the user's behalf.
- MCP token: An on-behalf-of token obtained through PingOne token exchange. The `act` and `sub` claims assert that this token is used by the agent on behalf of the test user.


```
None


Agent token (actor token):
{
  "aud": [
    "agent"
  ],
  "client_id": "adeeb901-7b64-453d-92bc-059bcbcc5958",
  "env": "4631af52-7ec7-48cf-848f-48cf8ca8e1ab",
  "exp": 1774043329,
  "iat": 1774039729,
  "iss": "https://auth.pingone.com/4631af52-7ec7-48cf-848f-48cf8ca8e1ab/as",
  "jti": "722238d2-4748-4681-889d-7fe7a321b037",
  "org": "84c4c0c0-0ca2-43b4-a62c-d830882855bc",
  "p1.rid": "722238d2-4748-4681-889d-7fe7a321b037",
  "scope": "agent"
```

```
}

...

User token (subject token):
{
  "acr": "Agent-Consent-Login",
  "aud": [
    "agent"
  ],
  "auth_time": 1774039361,
  "client_id": "adeeb901-7b64-453d-92bc-059bcbcc5958",
  "env": "4631af52-7ec7-48cf-848f-48cf8ca8e1ab",
  "exp": 1774043333,
  "iat": 1774039733,
  "iss": "https://auth.pingone.com/4631af52-7ec7-48cf-848f-48cf8ca8e1ab/as",
  "jti": "3121e22a-5958-4bf1-a369-ba296b444930",
  "may_act": {
    "sub": "adeeb901-7b64-453d-92bc-059bcbcc5958"
  },
  "org": "84c4c0c0-0ca2-43b4-a62c-d830882855bc",
  "p1.userId": "cf025143-501a-4d70-9c80-95d69121d7b3",
  "scope": "agent",
  "sid": "2dd6d4a4-0b59-4c99-8a57-4c189f09b24f",
  "sub": "demouser"
}

[INFO] Exchanging actor token and subject token for a new mcp token
(scope='test')

Exchanged on-behalf-of token (mcp token):
{
  "acr": "Agent-Consent-Login",
  "act": {
    "sub": "adeeb901-7b64-453d-92bc-059bcbcc5958"
  },
  "aud": [
    "https://ig.example.com:8443/mcp"
  ],
  "auth_time": 1774039361,
  "client_id": "adeeb901-7b64-453d-92bc-059bcbcc5958",
  "env": "4631af52-7ec7-48cf-848f-48cf8ca8e1ab",
  "exp": 1774043333,
  "iat": 1774039733,
```

```
  "iss": "https://auth.pingone.com/4631af52-7ec7-48cf-848f-48cf8ca8e1ab/as",
  "jti": "e1e90599-b09f-4549-8ea9-6a297490e53c",
  "org": "84c4c0c0-0ca2-43b4-a62c-d830882855bc",
  "p1.userId": "cf025143-501a-4d70-9c80-95d69121d7b3",
  "scope": "test",
  "sid": "2dd6d4a4-0b59-4c99-8a57-4c189f09b24f",
  "sub": "demouser"
}
```


4. Check the PingGateway log for additional details about the MCP request.

   Result: You have successfully validated that PingGateway can protect the MCP server.

Troubleshooting

Token exchange fails / `act` claim is null

Verify that the attribute expressions in your custom resources accurately match client IDs and that the `act` claim is properly mapped in the token.

PingGateway drops MCP traffic

Verify that `streamingEnabled` is set to `true` in your `admin.json` file. PingGateway requires this setting for the Server-Sent Events (SSE) utilized by the MCP protocol.

Gateway routing errors

Verify that the `UriPathRewriteFilter` is properly stripping the `/mcp` path down to `/` before it reaches the backend MCP server, as most MCP servers expect requests at the root.

What’s next

Add fine-grained authorization policies to secure agent actions using real-time contextual signals. Learn more about authorization with PingOne Authorize.